Web Technologies

CSS

Summary of Today's Lecture

- CSS Introduction
- CSS Rules
- CSS Selectors
  - Simple Selectors
  - Pseudo-class Selectors
  - Attribute Selectors
  - Combinator Selectors
- Inserting Style Sheet

CSS

- CSS stands for cascading style sheets.
- **Cascading:** refers to the procedure that determines which style will apply to a certain section, if you have more than one style rules.
- **Style:** how you want certain part of your page to look like. You can set things like color, margins, fonts etc, for things like tables, paragraphs, headings etc.
- **Sheets:** are like templates, or a set of rules, for determining how the webpage will look.
- CSS is a stylesheet language used to describe the presentation of a document written in HTML.

CSS History

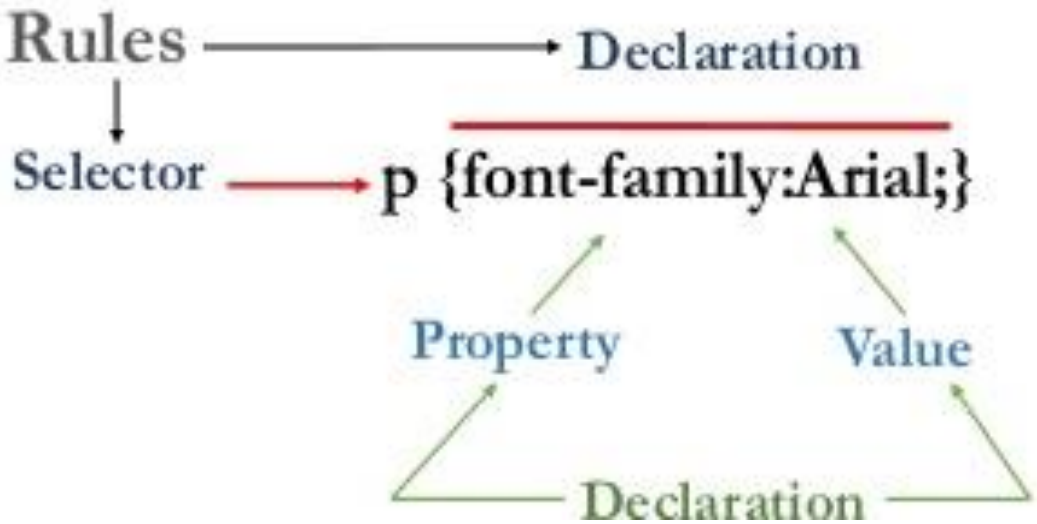
- CSS was first proposed by Håkon Wium Lie on October 10, 1994. At the time, Lie was working with Tim Berners-Lee at CERN.
- CSS1 was the first edition introduced in December 1996 an official W3C Recommendation. Håkon Wium Lie and Bert Bos are credited as the original developers.
- CSS2 was published by W3C in 1998 and provides enhancement over CSS1.
- CSS3 is the latest edition. The earliest CSS 3 drafts were published in June 1999 by W3C.
  - Several new functionalities have been provided through CSS3. Functions like flex box, rounded corners, background decoration, box shadows are introduced in this version.
- There is no single, integrated CSS4 specification, because the specification has been split into many separate modules which level independently.
  - So far, five such "best current practices" documents have been published as Notes, in 2007, 2010, 2015, 2017, and 2018.

CSS Rules

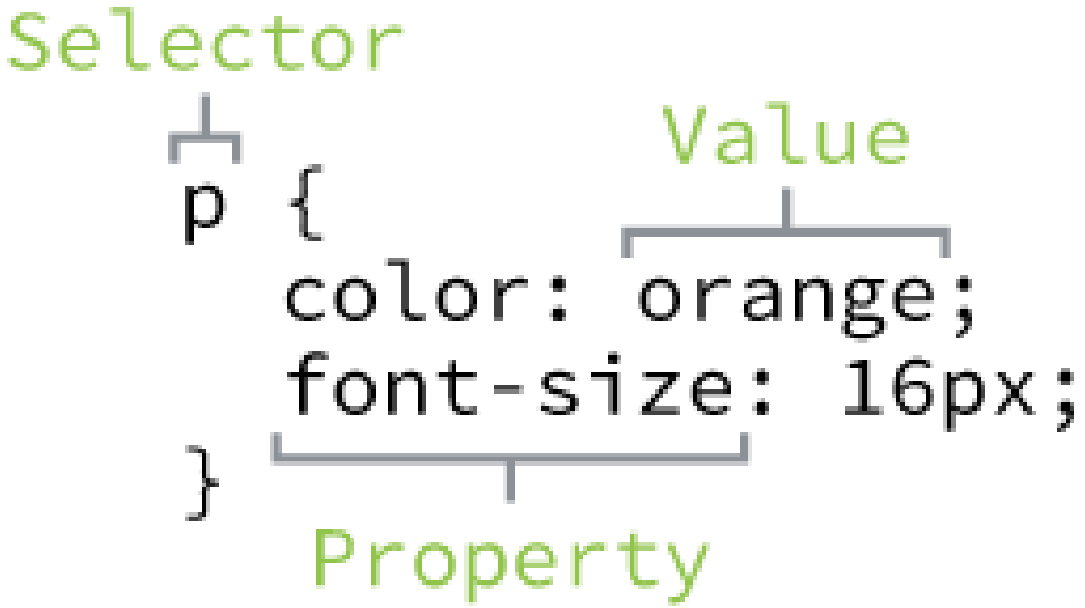
Rule have two parts - Selector and declaration.

Selector: The HTML element you want to add style to.
<p> <h1> <table> etc

Declaration: The statement of style for that element. Made up of property and value.



Selector



- **Selector**
  - A *selector* designates exactly which element or elements within our HTML to target and apply styles to.
    - `p { ... }`

Property and Value

- **Property**
  - A property determines the styles that will be applied to the element.
  - Property names fall after a selector, within the curly brackets, {}, and immediately preceding a colon, :.
  - There are numerous properties we can use, such as `background`, `color`, `font-size`, `height`, and `width`,
    - `p { color: ...; font-size: ...; }`
- **Value**
  - Behavior of the property is determined with a value. Values can be identified as the text between the colon, :, and semicolon, ;.
    - `p { color: orange; font-size: 16px; }`

CSS Selectors

- CSS selectors are used to "find" (or select) the HTML elements you want to style.
- We can divide CSS selectors into five categories:
  - Simple selectors (select elements based on name, id, class)
  - Pseudo-class selectors (select elements based on a certain state)
  - Pseudo-elements selectors (select and style a part of an element)
  - Attribute selectors (select elements based on an attribute or attribute value)
  - Combinator selectors (select elements based on a specific relationship between them)

The CSS Universal Selector

- The universal selector (*) selects all HTML elements on the page.
- The CSS rule below will affect every HTML element on the page:

```
<html>
<head>
<style>
* {
  text-align: center;
  color: blue;
}
</style>
</head>
<body>

<h1>Hello world!</h1>

<p>Every element on the page will be affected by the style.</p>
<p>Me too!</p>
<p>And me!</p>

</body>
</html>
```



CSS element Selector

- The element selector selects HTML elements based on the element name.
- For example, if we want to target all division elements of type `<div>`, we will use element selector of `div`.

- **HTML**
  - `<div>...</div>`
  - `<p>...</p>`
  - `<div>...</div>`
  - `<p>...</p>`
- **CSS**
  - `div { ... }`

CSS class Selector

- *Class* selectors allow us to select an element based on the element's `class` attribute value.
- Class selectors are a little more specific than type selectors, as they select a group of elements rather than all elements of one type.
- Within CSS, classes are denoted by a leading period, `.`, followed by the class attribute value.

- In the example below, the class selector will select any element containing the class attribute value of ***awesome***, including both `div` and `p` elements.
- **HTML**
  - `<div class="awesome">...</div>`
  - `<p>...</p>`
  - `<div>...</div>`
  - `<p class="awesome">...</p>`
- **CSS**
  - `.awesome { ... }`

CSS id Selector

- ID selectors are even more precise than `class` selectors, as they target only one unique element at a time.
- ID selectors use an element's `id` attribute value as a selector.
- ID attribute values can only be used once per page.
- Within CSS, ID selectors are denoted by a leading hash sign, `#`, followed by the `id` attribute value.

- In the example below, ID selector will only select the element containing the `id` attribute value of ***shayhowe*** and ***para***.
- **HTML**
  - `<div id="shayhowe">...</div>`
  - `<p>...</p>`
  - `<div>...</div>`
  - `<p id="para">...</p>`
- **CSS**
  - `#shayhowe { ... }`
  - `#para{ ... }`

Inserting a Stylesheet

- When a browser reads a style sheet, it will format the HTML document according to the information in the style sheet.
- There are *three ways of inserting a style sheet:*
  - Inline style
  - Internal style sheet
  - External style sheet

Inline Styles- the *style* Attribute

- An inline style may be used to apply a unique style for a single element.
- To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.
- An Inline style sheet loses many of the advantages of a style sheet. This is done by mixing content with presentation.
- Used for one-time overrides and styling a particular element.
- This is *bad style and should be avoided when possible.*

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>
```



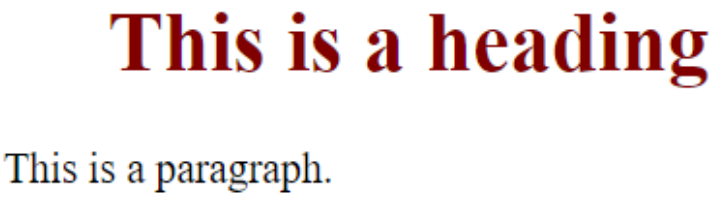
Internal Style Sheets

- An internal style sheet may be used if one single HTML page has a unique style.
- Internal styles are placed in the `<head>` section of an HTML page.
- Internal styles are inserted inside the `<style>` tag.

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```



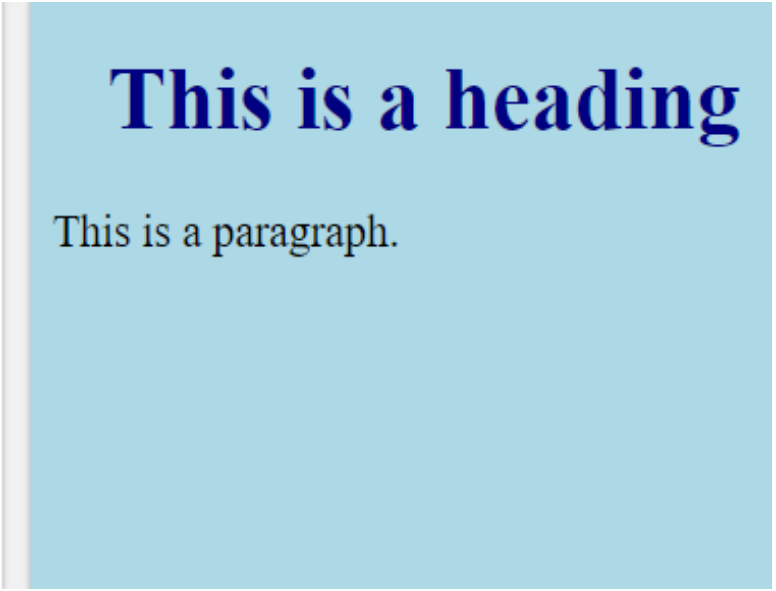
External Style sheet

- An external style sheet is ideal when the style is applied to many pages.
- With an external style sheet, you can change the *look of an entire Web site by changing just one file.*
- Each page must include a *link to the style sheet with the* `<link>` *tag.*
- An external style sheet can be written in any text editor and must be saved with a `.css` extension.
- A page can link to multiple style sheet files.
  - In case of a conflict (two sheets define a style for the same HTML element), the later sheet's properties will be used.

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```



Why CSS is called Cascading?

- Multiple style sheets can be referenced inside the HTML document.
- Which styles will be applicable when there is more than one style specified?
- All styles cascade into a new virtual style sheet *in this* order (with lowest to highest priority in the sequence as described below):
  - Browser's default styles
  - External style sheet files (in a `<link>` tag)
  - Internal style sheets (inside a `<style>` tag in the page's header)
  - Inline style (the `style` attribute of the HTML element)

Why CSS is called Cascading?

- In a single stylesheet, all styles cascade from the top of a style sheet to the bottom, allowing different styles to be added or overwritten as the style sheet progresses.
- For example, we select all paragraph elements at the top of our style sheet and set their background color to `orange` and their font size to `24 pixels`.
- Then towards the bottom of our style sheet, we select all paragraph elements again and set their background color to `green`.

- `p {`
  - `background: orange;`
  - `font-size: 24px; }`
- `p {`
  - `background: green;}`
- Because the paragraph selector that sets the background color to green comes after the paragraph selector that sets the background color to orange, it will take precedence in the cascade.
- All of the paragraphs will appear with a green background.
- The font size will remain 24 pixels because the second paragraph selector didn't identify a new font size.

Selector Specificity (Example)

- The higher the specificity weight of a selector, the more superiority the selector is given when a styling conflict occurs.
- For example, if a paragraph element is selected using a `type` selector in one place and an `ID` selector in another, the `ID` selector will take precedence over the `type` selector regardless of where the `ID` selector appears in the cascade.
- **HTML**
  - `<p id="food">...</p>`
- **CSS**
  - `#food { background: green; }`
  - `p { background: orange; }`

- **ID selector specificity > Class > Type**

Pseudo-class Selectors

- Pseudo-classes are similar to regular classes in HTML however they are not directly stated within the HTML markup.
- They are dynamically populated as a result of user's actions or the document structure.
- They are used to define a special state of an element. For example, it can be used to:
  - Style an element when a user moves mouse over it.
  - Style visited and unvisited links differently.
  - Style an element when it gets focus.
- **Syntax:**
  - `selector:pseudo-class { property: value; }`
- Types:
  - Link - Pseudo-classes
  - User Action - Pseudo-classes
  - User Interface State - Pseudo-classes

Link Pseudo-classes

- The `:link` and `:visited` pseudo-classes define if a link has or hasn't been visited.
- To style an anchor which has not been visited the `:link` pseudo-class is used.
- The `:visited` pseudo-class styles links that a user has already visited based on their browsing history.

- **Syntax:**
  - `a:link {...}`
  - `a:visited {...}`
- **Example:**
  - /* unvisited link */

```
a:link {
  color: #FF0000;
}
```

  - /* visited link */

```
a:visited {
  color: #00FF00;
}
```

User Action Pseudo-classes

- The `:hover` pseudo-class is applied to an element when a user moves their cursor over the element, most commonly used with anchor elements.
- The `:active` pseudo-class is applied to an element when a user engages an element, such as clicking on an element.
- Lastly, the `:focus` pseudo-class is applied to an element when a user has made an element the focus point of the page, often by using the keyboard to tab from one element to another.

- **Syntax**
  - `a:hover {...}`
  - `a:active {...}`
  - `a:focus {...}`

- **Example**

  - /* mouse over link */
    ```
    a:hover {
      color: #FF00FF;
    }
    ```

  - /* selected link */
    ```
    a:active {
      color: #0000FF;
    }
    ```

User Interface State Pseudo-classes

- These user interface element state pseudo-classes include
  - `:enabled`,
  - `:disabled`,
  - `:checked`.
- The `:enabled` pseudo-class selects an input that is in the default state of enabled and available for use.
- `:disabled` pseudo-class selects an input that has the disabled attribute tied to it.
- `:checked r`epresents the selected states of radio and checkbox elements.

- **Syntax:**
  - `input:enabled {...}`
  - `input:disabled {...}`
- **Example:**
  - `input[type="text"]:enabled { background-color:red; }`
  - `input[type="text"]:disabled { background-color:blue; }`
  - `input:checked { border:1px solid pink;}`

Pseudo-classes and CSS Classes

- Pseudo-classes can be combined with CSS classes:
- When you hover over the link in the example, it will change color:

```
<!DOCTYPE html>
<html>
<head>
<style>
a.highlight:hover {
  color: #ff0000;
}
</style>
</head>
<body>

<p><a class="highlight" href="css_syntax.asp">CSS Syntax</a></p>
<p><a href="default.asp">CSS Tutorial</a></p>

</body>
</html>
```

CSS Syntax

CSS Tutorial

Miscellaneous Pseudo-classes

All CSS Pseudo Classes

Selector	Example	Example description
:active	a:active	Selects the active link
:checked	input:checked	Selects every checked <input> element
:disabled	input:disabled	Selects every disabled <input> element
:empty	p:empty	Selects every <p> element that has no children
:enabled	input:enabled	Selects every enabled <input> element
:first-child	p:first-child	Selects every <p> elements that is the first child of its parent
:first-of-type	p:first-of-type	Selects every <p> element that is the first <p> element of its parent
:focus	input:focus	Selects the <input> element that has focus
:hover	a:hover	Selects links on mouse over
:in-range	input:in-range	Selects <input> elements with a value within a specified range
:invalid	input:invalid	Selects all <input> elements with an invalid value

Attribute Selectors

- It is possible to style HTML elements that have specific attributes or attribute values.
- In CSS3, now elements can be selected based on whether an attribute is present and what its value may contain.
- Following are the list of attribute selectors available in CSS.
  - Attribute Present Selector
  - Attribute Equals Selector
  - Attribute Contains Selector
  - Attribute Begins With Selector
  - Attribute Ends With Selector

Attribute Present Selector

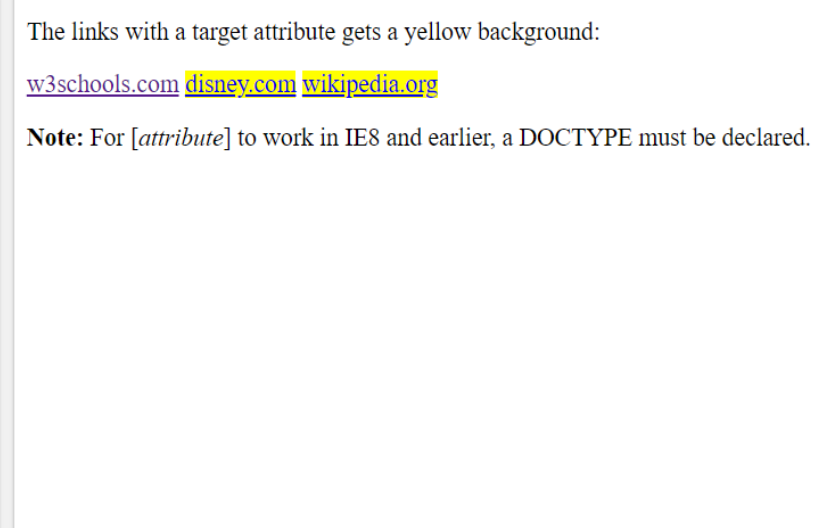
- The first attribute selector identifies an element based on whether it includes an attribute or not, regardless of any actual value.
- To select an element based on if an attribute is present or not simply include the attribute name in square brackets, [], within a selector.
- The `[attribute]` selector is used to select elements with a specified attribute.
- The following example selects all `<a>` elements with a target attribute:

```
<!DOCTYPE html>
<html>
<head>
<style>
a[target] {
  background-color: yellow;
}
</style>
</head>
<body>

<p>The links with a target attribute gets a yellow background:</p>

<a href="https://www.w3schools.com">w3schools.com</a>
<a href="http://www.disney.com" target="_blank">disney.com</a>
<a href="http://www.wikipedia.org" target="_top">wikipedia.org</a>

<p><b>Note:</b> For [<i>attribute</i>] to work in IE8 and earlier, a DOCTYPE must
be declared.</p>
```



Attribute Equals Selector

- To identify an element with a specific, and exact matching, attribute value the same selector from before may be used, however this time inside of the square brackets following the attribute name, include the desired matching value.
- The `[attribute="value"]` selector is used to select elements with a specified attribute and value.
- The following example selects all `<a>` elements with a `target="_blank"` attribute:

```
<!DOCTYPE html>
<html>
<head>
<style>
a[target=_blank] {
  background-color: yellow;
}
</style>
</head>
<body>

<p>The link with target="_blank" gets a yellow background:</p>

<a href="https://www.w3schools.com">w3schools.com</a>
<a href="http://www.disney.com" target="_blank">disney.com</a>
<a href="http://www.wikipedia.org" target="_top">wikipedia.org</a>

<p><b>Note:</b> For [<i>attribute</i>] to work in IE8 and earlier,
a DOCTYPE must be declared.</p>
```

The link with target="_blank" gets a yellow background:

w3schools.com disney.com wikipedia.org

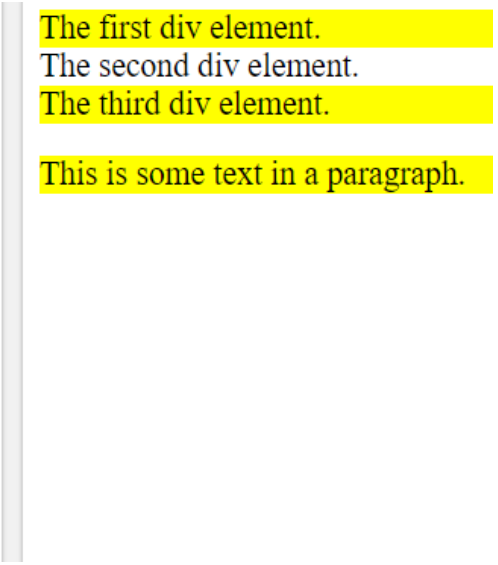
Note: For [*attribute*] to work in IE8 and earlier, a DOCTYPE must be declared.

Attribute Contains Selector

- When looking to find an element based on part of an attribute value, but not an exact match, the asterisk character, *, may be used within the square brackets of a selector.
- The asterisk should fall just after the attribute name, directly before the equals sign.
- The `[attribute*="value"]` selector is used to select elements whose attribute value contains a specified value.
- The following example selects all elements with a class attribute value that contains "`te`":

```
<!DOCTYPE html>
<html>
<head>
<style>
[class*="te"] {
  background: yellow;
}
</style>
</head>
<body>

<div class="first_test">The first div element.</div>
<div class="second">The second div element.</div>
<div class="my-test">The third div element.</div>
<p class="mytest">This is some text in a paragraph.</p>
```



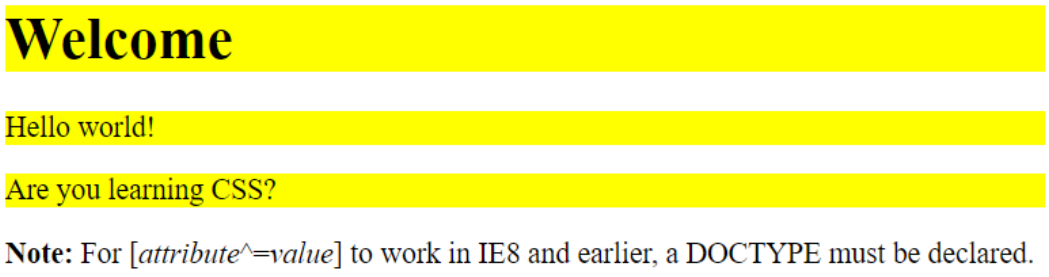
Attribute Begins With Selector

- It is also possible to select an element based on what an attribute value begins with. Using a circumflex accent (caret), ^, within the square brackets of a selector between the attribute name and equals sign denotes that the attribute value should begin with the stated value.
- The `[attribute^="value"]` selector is used to select elements whose attribute value begins with a specified value.
- The following example selects all elements with a class attribute value that begins with "`top`":

```
<!DOCTYPE html>
<html>
<head>
<style>
[class^="top"] {
  background: yellow;
}
</style>
</head>
<body>

<h1 class="top-header">Welcome</h1>
<p class="top-text">Hello world!</p>
<p class="topcontent">Are you learning CSS?</p>

<p><b>Note:</b> For [<i>attribute</i>^=
<i>value</i>] to work in IE8 and earlier, a
DOCTYPE must be declared.</p>
```

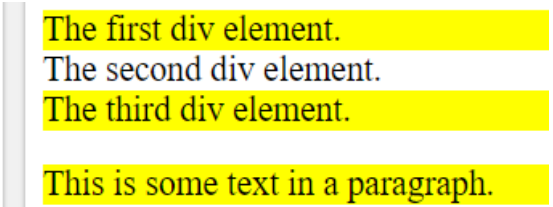


Attribute Ends With Selector

- There is also an ends with attribute selector. Instead of using the circumflex accent, the ends with attribute selector uses the dollar sign, $, within the square brackets of a selector between the attribute name and equals sign. Using the dollar sign denotes that the attribute value needs to end with the stated value.
- The `[attribute$="value"]` selector is used to select elements whose attribute value ends with a specified value.
- The following example selects all elements with a class attribute value that ends with "test":

```
<!DOCTYPE html>
<html>
<head>
<style>
[class$="test"] {
  background: yellow;
}
</style>
</head>
<body>

<div class="first_test">The first div element.</div>
<div class="second">The second div element.</div>
<div class="my-test">The third div element.</div>
<p class="mytest">This is some text in a paragraph.</p>
```



Attribute Selectors Overview

All CSS Attribute Selectors

Selector	Example	Example description
[*attribute*]	[target]	Selects all elements with a target attribute
[*attribute=value*]	[target=_blank]	Selects all elements with target="_blank"
[*attribute~=value*]	[title~=flower]	Selects all elements with a title attribute containing the word "flower"
[*attribute|=value*]	[lang|=en]	Selects all elements with a lang attribute value starting with "en"
[*attribute^=value*]	a[href^="https"]	Selects every <a> element whose href attribute value begins with "https"
[*attribute$=value*]	a[href$=".pdf"]	Selects every <a> element whose href attribute value ends with ".pdf"
[*attribute*=value*]	a[href*="w3schools"]	Selects every <a> element whose href attribute value contains the substring "w3schools"

Combining Selectors

- A CSS selector can contain more than one simple selector. Between the simple selectors, we can include a combinator.
- A combinator is something that explains the relationship between the selectors.
- By combining selectors we can be more specific about which element or group of elements we'd like to select.
  - Child Selectors
  - Sibling Selectors

Child Selectors

- Child selectors provide a way to select elements that fall within one another, thus making them children of their parent element.
- These selections can be made two different ways,
  - Descendant selector
  - Direct child selectors.

Descendant Selector

- The descendant selector matches all elements that are descendants of a specified element.
- Descendant selectors are created by spacing apart elements within a selector, creating a new level of hierarchy for each element list.
- The following example selects all <p> elements inside elements:

```
<!DOCTYPE html>
<html>
<head>
<style>
div p {
  background-color: yellow;
}
</style>
</head>
<body>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section>
</div>

<p>Paragraph 4. Not in a div.</p>
<p>Paragraph 5. Not in a div.</p>

</body>
</html>
```

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div.

Paragraph 4. Not in a div.

Paragraph 5. Not in a div.

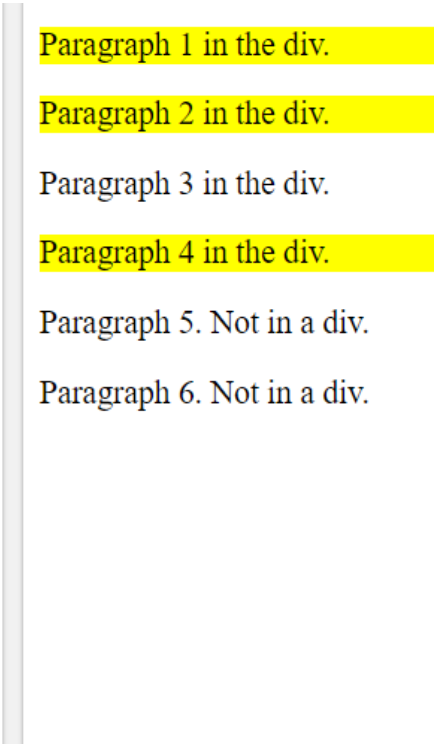
Direct Child Selector

- The child selector selects all elements that are the children of a specified element.
- The direct child selector may be used by placing a greater than sign, >, between the parent element and child element within the selector.
- The following example selects all <p> elements that are children of a element:

```
<!DOCTYPE html>
<html>
<head>
<style>
div > p {
  background-color: yellow;
}
</style>
</head>
<body>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section> <!-- not Child but Descendant -->
  <p>Paragraph 4 in the div.</p>
</div>

<p>Paragraph 5. Not in a div.</p>
<p>Paragraph 6. Not in a div.</p>
```



Child Selector Overview

Example	Classification	Explanation
`article h2`	Descendant Selector	Selects an element that resides anywhere within an identified ancestor element
`article > p`	Direct Child Selector	Selects an element that resides immediately inside an identified parent element

Sibling Selectors

- Sibling elements are those elements that share a common parent, may also need to be selected.
- These sibling selections can be made by two ways
  - General sibling
  - Adjacent sibling selectors.

General Sibling Selector

- The general sibling selector selects all elements that are the **next** siblings of a specified element.
- They are created by using the tilde character, ~, between two elements within a selector. The first element identifies what the second element shall be a sibling with, and both of which must share the same parent.
- The following example selects all <p> elements that are the next siblings of elements:

```
<!DOCTYPE html>
<html>
<head>
<style>
div ~ p {
  background-color: yellow;
}
</style>
</head>
<body>

<p>Paragraph 1.</p>

<div>
  <p>Paragraph 2.</p>
</div>

<p>Paragraph 3.</p>
<code>Some code.</code>
<p>Paragraph 4.</p>
```

Paragraph 1.

Paragraph 2.

Paragraph 3.

Some code.

Paragraph 4.

Adjacent Sibling Selector

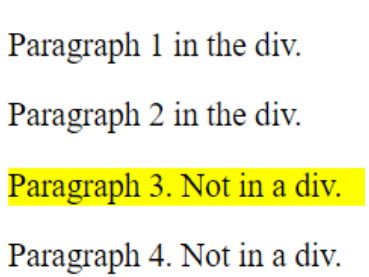
- The adjacent sibling selector selects all elements that are the adjacent siblings of a specified element.
- Sibling elements must have the same parent element, and "adjacent" means "immediately following".
- Adjacent sibling selector uses a plus character, +, between the two elements within a selector.
- The following example selects all <p> elements that are placed immediately after elements:

```
<!DOCTYPE html>
<html>
<head>
<style>
div + p {
  background-color: yellow;
}
</style>
</head>
<body>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
</div>

<p>Paragraph 3. Not in a div.</p>
<p>Paragraph 4. Not in a div.</p>

</body>
</html>
```



Sibling Selectors Overview

Example	Classification	Explanation
`h2 ~ p`	General Sibling Selector	Selects an element that follows anywhere after the prior element, in which both elements share the same parent
`h2 + p`	Adjacent Sibling Selector	Selects an element that follows directly after the prior element, in which both elements share the same parent

Summary of Today's Lecture

- CSS Introduction
- CSS Rules
- CSS Selectors
  - Simple Selectors
  - Pseudo-class Selectors
  - Pseudo-element Selectors
  - Attribute Selectors
  - Combinator Selectors
- Inserting Style Sheet

Resources

- The complete list of these selectors along with their description can be found at the following links:
  - https://www.w3.org/TR/css3-selectors/#checked
  - https://www.w3schools.com/css/css_pseudo_classes.asp
  - https://www.w3schools.com/cssref/css_selectors.asp
  - http://learn.shayhowe.com/advanced-html-css/complex-selectors/